Actividad 7:

MINI JUEGO "PIEDRA, PAPEL O TIJERA" CON PERSISTENCIA DE DATOS

Repositorio de GitHub

https://github.com/Martuuuuuuuu/MINIJUEGO.git

Participantes

Martina Araujo, Thiago Goya, Sofia Salaberry

Escuela

Escuela Técnica N°1 de Vicente López

Curso

Ciclo Superior 5°3° turno tarde

Docentes a cargo

Yamil Ganduglia, York Elias Mansilla Muñoz

Año:

2026

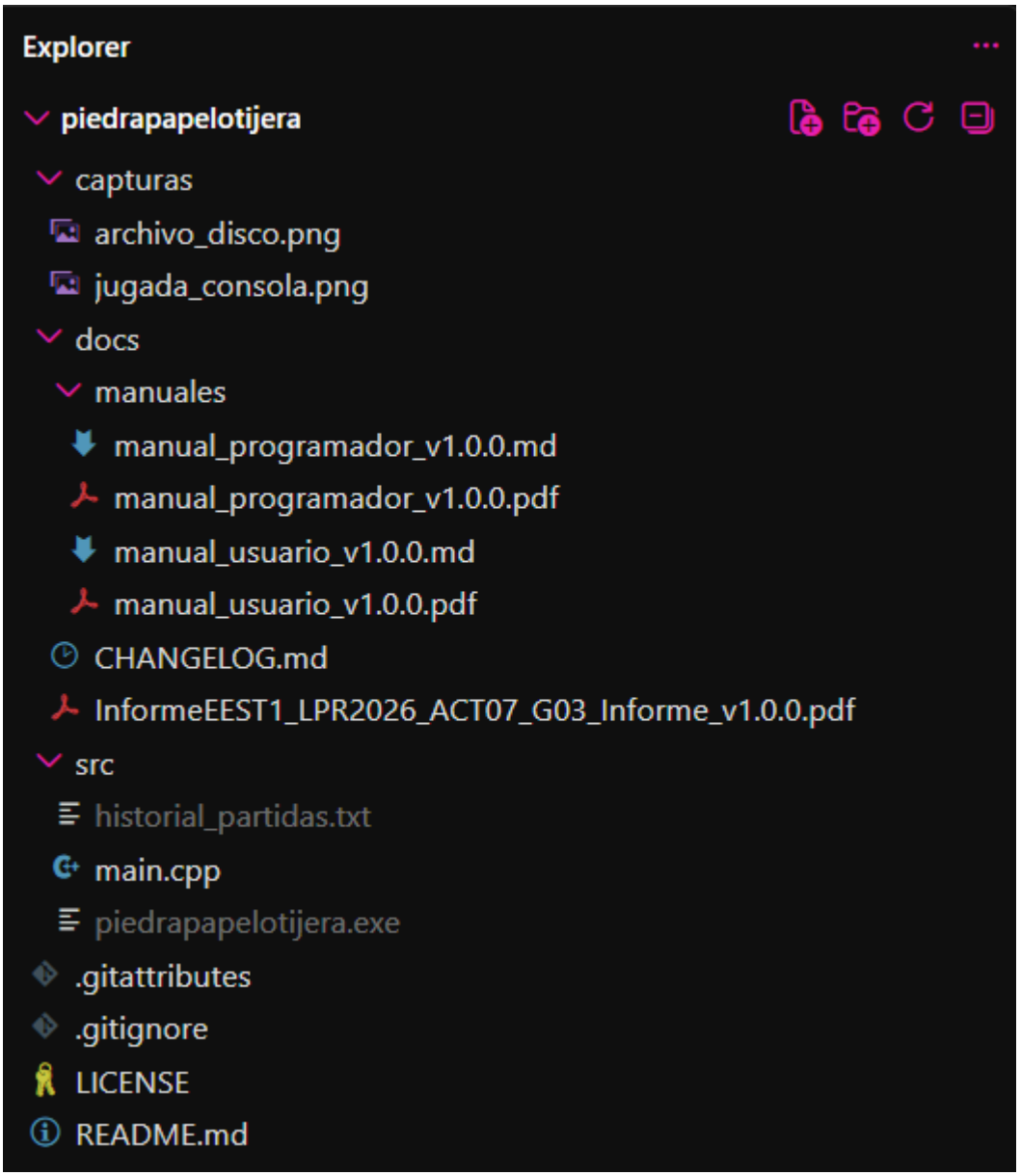
Tabla de Revisiones

Versión	Fecha	Descripción de Cambio	Autor/Integrante	Estado
v0.1.0	14/10/26	Borrador inicial del informe y código C++	Thiago Goya, Sofia Salaberry	Borrador
v0.2.0	15/10/26	Agregado de diagramas de RAM y capturas de consola	Martina Araujo, Sofia Salaberry	En Revisión
v1.0.0	16/10/26	Versión final corregida para la entrega oficial	Martina Araujo, Thiago Goya y Sofía Salaberry	Completado

Introducción

En esta actividad desarrollamos un Mini Juego de Piedra, Papel o Tijera contra la Inteligencia Artificial de la Consola, donde cada partida jugada se registrará automáticamente en una estructura (struct) y se guardará físicamente en un archivo de texto (historial_partidas.txt) en el disco rígido/estado sólido.

Estructura de Carpetas



Código Fuente

src > main.cpp

```cpp
/*
    ================================================================
    Materia: Laboratorio de Programación (LPR) – 5° Año
    Institución: E.E.S.T. N° 1 "Escuela de Educación Secundaria Técnica N° 1" – Vicente López
    Archivo: src/main.cpp
    Actividad 7: Mini Juego Piedra, Papel o Tijera + Persistencia en Disco
    ================================================================
*/

#include <iostream>
#include <fstream>
#include <string>
#include <cstdlib>
#include <ctime>

using namespace std;

// 1. Estructura de datos para representar una partida
struct Partida {
    int idPartida;
    string jugador;
    string eleccionJugador;
    string eleccionIA;
    string resultado;
};

// Prototipos de funciones modulares
void jugarPartida(int& contadorID);
void guardarEnDisco(const Partida& p);
void leerHistorialDisco();

int main() {
    // Inicializar la semilla para los números aleatorios de la IA
    srand(time(0));

    int contadorID = 1;
```

```
    int opcion = 0;

    cout << "=================================================" << endl;
    cout << "  LPR 2026 - ACTIVIDAD 7: PIEDRA, PAPEL O TIJERA      " << endl;
    cout << "  Estudiante / Grupo: [Martina Araujo]          " << endl;
    cout << "=================================================" << endl;

    do {
        cout << "\n--- MENU PRINCIPAL DE JUEGO Y PERSISTENCIA ---" << endl;
        cout << "1. Jugar Partida (Piedra, Papel o Tijera)" << endl;
        cout << "2. Ver Historial de Partidas Guardadas en Disco (.txt)" << endl;
        cout << "3. Salir del Sistema" << endl;
        cout << "Ingrese una opcion (1-3): ";
        cin >> opcion;

        if (opcion == 1) {
            jugarPartida(contadorID);
        }
        else if (opcion == 2) {
            leerHistorialDisco();
        }
        else if (opcion == 3) {
            cout << "\n[SISTEMA] Saliendo del programa... Datos resguardados en disco." << endl;
        }
        else {
            cout << "\n[ERROR] Opcion invalida. Reintente." << endl;
        }

    } while (opcion != 3);

    return 0;
}

// Función para procesar la jugada contra la IA
void jugarPartida(int& contadorID) {
    Partida p;
    p.idPartida = contadorID++;

    cout << "\n--- NUEVA PARTIDA #" << p.idPartida << " ---" << endl;
    cout << "Ingrese su Nombre / Nickname: ";
    cin.ignore(); // Limpieza de búfer
    getline(cin, p.jugador);

    int opcJugador = 0;
    cout << "\nElija su jugada:" << endl;
    cout << "1. Piedra" << endl;
    cout << "2. Papel" << endl;
    cout << "3. Tijera" << endl;
    cout << "Opcion (1-3): ";
    cin >> opcJugador;

    // Asignar selección del usuario
    if (opcJugador == 1) p.eleccionJugador = "Piedra";
    else if (opcJugador == 2) p.eleccionJugador = "Papel";
    else p.eleccionJugador = "Tijera";

    // Elección aleatoria de la IA (1 a 3)
    int opcIA = (rand() % 3) + 1;
    if (opcIA == 1) p.eleccionIA = "Piedra";
    else if (opcIA == 2) p.eleccionIA = "Papel";
    else p.eleccionIA = "Tijera";

    // Evaluar reglas del juego
    cout << "\n[JUGADA] " << p.jugador << " eligio: " << p.eleccionJugador << endl;
    cout << "[JUGADA] La IA eligio: " << p.eleccionIA << endl;

    if (p.eleccionJugador == p.eleccionIA) {
        p.resultado = "Empate";
    }
```

```
106     else if ((p.eleccionJugador == "Piedra" && p.eleccionIA == "Tijera") ||
107              (p.eleccionJugador == "Papel" && p.eleccionIA == "Piedra") ||
108              (p.eleccionJugador == "Tijera" && p.eleccionIA == "Papel")) {
109         p.resultado = "Victoria";
110     }
111     else {
112         p.resultado = "Derrota";
113     }
114
115     cout << "=> RESULTADO: " << p.resultado << "!" << endl;
116
117     // Persistir el resultado inmediatamente en el archivo
118     guardarEnDisco(p);
119 }
120
121 // Función para ESCRIBIR en el archivo de texto en disco
122 void guardarEnDisco(const Partida& p) {
123     // ios::app permite agregar al final sin borrar partidas anteriores
124     ofstream archivo("src/historial_partidas.txt", ios::app);
125
126     if (!archivo.is_open()) {
127         cerr << "[ERROR] No se pudo abrir el archivo para guardar la partida." << endl;
128         return;
129     }
130
131     // Guardado de datos separados por comas (Formato CSV)
132     archivo << p.idPartida << ","
133             << p.jugador << ","
134             << p.eleccionJugador << ","
135             << p.eleccionIA << ","
136             << p.resultado << endl;
137
138     archivo.close();
139     cout << "[DISCO] Partida resguardada con exito en 'historial_partidas.txt'." << endl;
140 }
141
142 // Función para LEER el historial persistido desde el archivo
143 void leerHistorialDisco() {
144     ifstream archivo("src/historial_partidas.txt");
145
146     if (!archivo.is_open()) {
147         cout << "\n[AVISO] Aun no existen partidas guardadas en disco." << endl;
148         return;
149     }
150
151     cout << "\n==================================================" << endl;
152     cout << "  HISTORIAL DE PARTIDAS RECUPERADO DESDE DISCO      " << endl;
153     cout << "==================================================" << endl;
154
155     string idStr, jugador, elJug, elIA, resultado;
156     int total = 0;
157
158     // Lectura secuencial delimitada por comas
159     while (getline(archivo, idStr, ',') &&
160            getline(archivo, jugador, ',') &&
161            getline(archivo, elJug, ',') &&
162            getline(archivo, elIA, ',') &&
163            getline(archivo, resultado)) {
164
165         total++;
166         cout << "Partida #" << idStr
167              << " | Jugador: " << jugador
168              << " | " << elJug << " vs " << elIA
169              << " | Resultado: " << resultado << endl;
170     }
171
172     archivo.close();
173
```

```
171
172     archivo.close();
173
174     if (total == 0) {
175         cout << "[AVISO] El archivo de historial esta vacio." << endl;
176     } else {
177         cout << "\n[SISTEMA] Total de registros leidos desde disco: " << total << endl;
178     }
179     cout << "==================================================" << endl;
180 }
```

Capturas de Ejecución

1. Jugar Partida (Piedra, Papel o Tijera)

```
PS C:\Users\Martina\Desktop\piedrapapelotijera> .\src\piedrapapelotijera.exe
Ingrese una opcion (1-3): 1

--- NUEVA PARTIDA #1 ---
Ingrese su Nombre / Nickname: MartuPRO

Elija su jugada:
1. Piedra
2. Papel
3. Tijera
Opcion (1-3): 3

[JUGADA] MartuPRO eligio: Tijera
[JUGADA] La IA eligio: Papel
=> RESULTADO: Victoria!
[DISCO] Partida resguardada con exito en 'historial_partidas.txt'.
```

2. Ver Historial de Partidas Guardadas en Disco (.txt)

```
PS C:\Users\Martina\Desktop\piedrapapelotijera> .\src\piedrapapelotijera.exe
1. Jugar Partida (Piedra, Papel o Tijera)
2. Ver Historial de Partidas Guardadas en Disco (.txt)
3. Salir del Sistema
Ingrese una opcion (1-3): 2

==================================================
  HISTORIAL DE PARTIDAS RECUPERADO DESDE DISCO
==================================================
Partida #1 | Jugador: MartuPRO | Tijera vs Tijera | Resultado: Empate
Partida #2 | Jugador: MartuPRO | Piedra vs Piedra | Resultado: Empate
Partida #3 | Jugador: MartuPRO | Papel vs Papel | Resultado: Empate
Partida #1 | Jugador: MartuPRO | Tijera vs Papel | Resultado: Victoria

[SISTEMA] Total de registros leidos desde disco: 4
==================================================
```

3. Salir del Sistema

```
PS C:\Users\Martina\Desktop\piedrapapelotijera> .\src\piedrapapelotijera.exe
  ==================================================
    LPR 2026 - ACTIVIDAD 7: PIEDRA, PAPEL O TIJERA
    Estudiante / Grupo: [Martina Araujo]
  ==================================================

  --- MENU PRINCIPAL DE JUEGO Y PERSISTENCIA ---
  1. Jugar Partida (Piedra, Papel o Tijera)
  2. Ver Historial de Partidas Guardadas en Disco (.txt)
  3. Salir del Sistema
  Ingrese una opcion (1-3): 3

  [SISTEMA] Saliendo del programa... Datos resguardados en disco.
PS C:\Users\Martina\Desktop\piedrapapelotijera>
```

4. Historial_partidas.txt

```
PS C:\Users\Martina\Desktop\piedrapapelotijera> .\src\piedrapapelotijera.exe
================================================
  LPR 2026 - ACTIVIDAD 7: PIEDRA, PAPEL O TIJERA
  Estudiante / Grupo: [Martina Araujo]
================================================ …
================================================
  HISTORIAL DE PARTIDAS RECUPERADO DESDE DISCO
================================================
Partida #1 | Jugador: MartuPRO | Tijera vs Tijera | Resultado: Empate
Partida #2 | Jugador: MartuPRO | Piedra vs Piedra | Resultado: Empate
Partida #3 | Jugador: MartuPRO | Papel vs Papel | Resultado: Empate
Partida #1 | Jugador: MartuPRO | Tijera vs Papel | Resultado: Victoria
Partida #1 | Jugador: Thiago | Papel vs Tijera | Resultado: Derrota
Partida #2 | Jugador: Sofia | Piedra vs Papel | Resultado: Derrota
Partida #3 | Jugador: Martina | Tijera vs Piedra | Resultado: Derrota

[SISTEMA] Total de registros leidos desde disco: 7
================================================
```

5. Conexión exitosa

```
PS C:\Users\Martina\Desktop\piedrapapelotijera> git remote -v
origin  https://github.com/Martuuuuuuuu/MINIJUEGO.git (fetch)
origin  https://github.com/Martuuuuuuuu/MINIJUEGO.git (push)
origin  https://gitlab.com/Martuuuuuuuu/MINIJUEGO.git (push)
PS C:\Users\Martina\Desktop\piedrapapelotijera>
```

CUESTIONARIO TÉCNICO DE CONTROL (OBLIGATORIO EN EL INFORME)

Respondan de forma clara las siguientes 5 preguntas en su informe APA v7:

1. **RAM vs. Disco:** ¿Por qué si cerras la ventana de consola y volves a abrir el ejecutable, la Opción 2 sigue mostrando el historial de partidas anteriores?

2. **Uso de ios::app:** ¿Qué sucedería con el archivo historial_partidas.txt si en la función guardarEnDisco utilizaras ofstream archivo("historial_partidas.txt"); sin incluir la opción ios::app?

3. **Manejo del Búfer (cin.ignore()):** ¿Por qué es necesario invocar cin.ignore() antes de leer el nombre del jugador con getline(cin, p.jugador)?

4. **Agrupación con struct:** ¿Qué ventajas ofrece la estructura Partida para organizar las variables del juego frente a pasar 5 variables sueltas a cada función?

5. **Aplicación al P.I.A.:** Si adaptaran este módulo de almacenamiento a su Proyecto Integrador Anual, ¿qué datos o eventos de su prototipo guardarían en el archivo de texto?

Respuestas:

1. El historial sigue apareciendo porque las partidas se guardan en el archivo historial_partidas.txt, que está almacenado en el disco. A diferencia de la RAM, los datos del disco no se borran al cerrar el programa.
2. Si no se utiliza ios::app, el archivo se sobrescribe cada vez que se guardan nuevas partidas, por lo que se perdería el historial anterior. Con ios::app, los nuevos datos se agregan al final del archivo.
3. Se utiliza cin.ignore() para eliminar el salto de línea que queda pendiente después de usar cin. Así, getline() puede leer correctamente el nombre completo del jugador, incluidos los espacios.
4. La estructura Partida permite agrupar los datos relacionados con una partida en una sola variable. Esto facilita organizar la información, pasarla entre funciones y mantener el código más ordenado y fácil de entender.
5. En nuestro Proyecto Integrador Anual guardariamos los productos reservados, sus cantidades, los datos de las reservas y los movimientos de dinero de la cooperadora. Esto nos permitiría conservar un registro de las operaciones realizadas y consultar la información posteriormente.